

PlayArena

Documentação Técnica e Funcional para TCC

Projeto: PlayArena

Objetivo: Plataforma para reserva e gerenciamento de quadras e espaços esportivos

Cidade inicial: Campo Mourão - PR

Perfis atendidos: Usuário, Proprietário e Administrador

Data da análise: 14/06/2026

Base da análise: código-fonte localizado em `frontend/` e `backend/`

Índice

1. Visão geral do projeto
 2. Arquitetura do sistema
 3. Tecnologias utilizadas
 4. Estrutura de pastas
 5. Explicação das telas
 6. Fluxo completo do usuário
 7. Fluxo completo do proprietário
 8. Fluxo completo do administrador
 9. Banco de dados
 10. APIs
 11. Segurança
 12. Funcionalidades implementadas
 13. Funcionalidades futuras
 14. Perguntas possíveis da banca
 15. Roteiro de apresentação de 15 minutos
 16. Manual de estudo
 17. Organização da apostila e PDF final
-

1. Visão Geral do Projeto

1.1 O que é o PlayArena

O PlayArena é uma plataforma web para descoberta, reserva e gerenciamento de quadras e espaços esportivos. A aplicação conecta usuários que desejam praticar esportes com proprietários que possuem arenas, quadras ou espaços esportivos disponíveis para locação.

O sistema possui três experiências principais:

- **Usuário:** pesquisa espaços esportivos, visualiza detalhes, escolhe horários, realiza reservas, acompanha o histórico e recebe notificações.
- **Proprietário:** cadastra espaços, envia documentação, gerencia horários, acompanha reservas, valida códigos de presença, visualiza indicadores e gera relatórios.
- **Administrador:** valida proprietários/documentos, gerencia usuários, gerencia espaços, acompanha reservas, aplica bloqueios, banimentos e monitora alertas operacionais.

1.2 Problema que resolve

O problema central é a dificuldade de centralizar reservas esportivas em uma cidade, especialmente quando o processo depende de contato manual por telefone, mensagens, planilhas ou redes sociais. Isso gera:

- conflito de horários;
- falta de visibilidade sobre disponibilidade;
- dificuldade para comparar preço e modalidade;
- ausência de histórico confiável de reservas;
- dificuldade para proprietários acompanharem demanda e faturamento;
- pouca rastreabilidade para administração da plataforma.

O PlayArena resolve esse problema criando uma ponte digital entre usuários e proprietários, com regras de reserva, controle de disponibilidade, notificações, validação documental e gestão administrativa.

1.3 Objetivos do sistema

- Permitir que usuários encontrem espaços esportivos em Campo Mourão - PR.
- Permitir reserva de horários disponíveis.
- Centralizar o gerenciamento de quadras por proprietários.
- Validar proprietários e documentações antes da exposição pública dos espaços.
- Manter histórico de reservas e notificações.
- Dar ao administrador uma visão geral da operação.
- Oferecer indicadores para proprietários por meio de uma área de BI.
- Preparar a plataforma para evolução com pagamentos, avaliações e expansão geográfica.

1.4 Benefícios para usuários

- Pesquisa de quadras por modalidade, preço e localização.
- Visualização de detalhes, fotos, endereço, comodidades e horários.
- Reserva digital com código de confirmação.
- Histórico de reservas.
- Cancelamento com regras de reembolso.
- Notificações sobre cancelamentos, bloqueios e promoções.
- Mapa com espaços localizados em Campo Mourão.

1.5 Benefícios para proprietários

- Cadastro de espaços com fotos, modalidades, preço e horários.
- Envio de documentação para validação.
- Gerenciamento de horários fixos ou datas específicas.
- Visualização de reservas recebidas.
- Validação de presença por código de reserva.
- Cancelamento com justificativa.
- Indicadores de faturamento, demanda, ticket médio, horários mais fortes e desempenho por arena.
- Geração de relatório em PDF pela tela de BI usando impressão do navegador.

1.6 Benefícios para administradores

- Painel com indicadores gerais da plataforma.
 - Validação de documentação de espaços.
 - Aprovação/reprovação de proprietários.
 - Gerenciamento de usuários e proprietários.
 - Bloqueio temporário, banimento, desbanimento e exclusão permanente.
 - Gestão de espaços cadastrados.
 - Auditoria de ações administrativas.
 - Acompanhamento de alertas quando proprietários acumulam advertências.
-

2. Arquitetura do Sistema

2.1 Visão geral da arquitetura

O PlayArena utiliza uma arquitetura web cliente-servidor dividida em:

- **Frontend:** aplicação React criada com Vite.
- **Backend:** API REST construída com Node.js e Express.
- **Banco de dados:** PostgreSQL acessado pelo Sequelize ORM.
- **Armazenamento local de arquivos:** uploads de fotos e documentos em `backend/uploads/`.
- **Serviços externos:** ViaCEP, OpenStreetMap/Nominatim, provedores de e-mail e provedores opcionais de SMS.

2.2 Fluxo Frontend -> Backend -> Banco de Dados

1. O usuário interage com uma tela React.
2. A tela chama funções do hook central `usePlayArenaApp`.
3. O hook chama o adaptador de API em `frontend/src/services/playarenaApi.js`.
4. O adaptador usa `frontend/src/services/httpClient.js` para enviar requisições HTTP.
5. O backend recebe a requisição em uma rota Express.
6. A rota aciona o controlador correspondente.
7. O controlador chama um serviço de negócio.
8. O serviço consulta ou grava dados usando modelos Sequelize.
9. O Sequelize executa comandos SQL no PostgreSQL.
10. O backend retorna JSON ao frontend.
11. O frontend atualiza a interface e, quando necessário, exibe toast, modal ou notificação.

2.3 Diagrama textual da arquitetura

```

Usuário / Proprietário / Administrador
    |
    v
Frontend React + Vite
    |
    | HTTP REST + JSON / FormData
    v
Backend Node.js + Express
    |
    | Controllers -> Services -> Models
    v
Sequelize ORM + PostgreSQL/Neon
    |
    v
Dados persistidos
  
```

Serviços auxiliares:

```

Frontend <-> Backend SSE (/api/events)
Backend -> ViaCEP
Backend -> Nominatim/OpenStreetMap
Backend -> E-mail SMTP/Gmail/Resend
Backend -> Uploads locais
  
```

2.4 Como as APIs se comunicam

A comunicação principal é REST. O frontend envia requisições para a base `http://localhost:3333/api` por padrão. Essa URL é definida em `frontend/src/services/httpClient.js` e pode ser sobrescrita por variável de ambiente `VITE_API_URL`.

Exemplos:

- `POST /api/auth/login` para autenticação.
- `GET /api/quadras` para listar quadras públicas.

- `POST /api/reservas` para criar reserva.
- `PATCH /api/admin/documentacoes/:id/status` para aprovar ou reprovar documentação.

Para uploads, o frontend envia `FormData` :

- fotos de perfil em `/api/auth/profile` ;
- fotos de quadras em `/api/quadras/fotos` ;
- documentos de proprietários em `/api/quadras/documentos` .

2.5 Como funciona a autenticação

A autenticação usa JWT. O usuário informa e-mail, senha e opcionalmente o perfil. O backend valida a senha com `bcrypt` , verifica o status da conta e emite um token JWT assinado com `JWT_SECRET` .

O token contém:

```
sub: id da conta autenticada
perfil: usuario | proprietario | admin
```

O token é enviado pelo frontend no cabeçalho:

```
Authorization: Bearer <token>
```

No backend, o middleware `backend/src/middleware/auth.js` valida o token, carrega a conta correta no banco e injeta `request.auth` na requisição.

2.6 Como funciona o armazenamento dos dados

Os dados estruturados ficam no PostgreSQL por meio dos modelos Sequelize. Entre as tabelas principais estão:

- `usuarios` ;
- `proprietarios` ;
- `administradores` ;
- `quadras` ;
- `horarios_disponiveis` ;
- `reservas` ;
- `documentacoes_locais` ;
- `notificacoes` ;
- `logs_administrativos` ;
- `advertencias_proprietarios` ;
- `avisos_administrativos` .

Arquivos enviados ficam em diretórios locais:

- `backend/uploads/profile-photos` ;
- `backend/uploads/court-photos` ;
- `backend/uploads/documentos` .

O banco armazena o caminho/URL desses arquivos, e o Express expõe a pasta `uploads` como conteúdo estático.

2.7 Atualização em tempo real

O projeto não utiliza Socket.io. A atualização em tempo real é feita por **Server-Sent Events (SSE)** na rota:

```
GET /api/events
```

O backend envia eventos como:

- `connected` ;
- `data-changed` ;
- `account-access-changed` .

O frontend escuta esses eventos em `frontend/src/services/httpClient.js` e atualiza listas de quadras, reservas, notificações e status de acesso.

3. Tecnologias Utilizadas

Tecnologia	O que é	Para que serve	Por que foi utilizada	Onde está no projeto
React	Biblioteca JavaScript para construção de interfaces	Criar telas reativas e componentes reutilizáveis	Permite SPA com atualização dinâmica da interface	frontend/src/
Vite	Ferramenta de build e servidor de desenvolvimento frontend	Rodar e empacotar o React	É rápido, moderno e simples para projetos React	frontend/package.json , frontend/vite.config.js
JavaScript/JSX	Linguagem usada no frontend e backend	Implementar lógica, telas, serviços e API	Padroniza o projeto em uma linguagem principal	frontend/src/ , backend/src/
Node.js	Ambiente JavaScript no servidor	Executar o backend	Permite criar APIs com JavaScript	backend/src/server.js
Express	Framework web para Node.js	Criar rotas HTTP, middlewares e API REST	Simples e amplamente usado para APIs	backend/src/app/createApp.js , backend/src/routes/
Sequelize	ORM para Node.js	Mapear tabelas do banco em modelos JavaScript	Facilita consultas, relacionamentos e migrações simples	backend/src/models/ , backend/src/services/
PostgreSQL	Banco de dados relacional	Persistir usuários, reservas, quadras, notificações e logs	Suporta relacionamentos, transações, índices e JSONB	Configurado em backend/src/config/database.js
Neon DB	Plataforma serverless para PostgreSQL	Hospedar PostgreSQL em nuvem	Compatível com PostgreSQL e conexão por DATABASE_URL com SSL	Usado pela configuração do Sequelize
JWT	Token assinado para autenticação	Manter sessão stateless no backend	Evita sessão em memória e facilita proteção de rotas	backend/src/middleware/auth.js
bcrypt	Biblioteca de hash de senhas	Armazenar senhas e códigos de forma segura	Senhas nunca ficam salvas em texto puro	backend/src/services/authService.js
Multer	Middleware para upload de arquivos	Receber fotos e documentos	Integra upload multipart ao Express	backend/src/middleware/*Upload.js
CORS	Política de acesso entre origens	Permitir frontend acessar backend em portas diferentes	Necessário em ambiente local e produção separada	backend/src/app/createApp.js
Nodemailer	Envio de e-mails por SMTP/Gmail	Mandar códigos de verificação e recuperação	Permite confirmar cadastro e recuperar senha	backend/src/services/emailService.js
Resend API	Serviço de envio de e-mail	Alternativa moderna ao SMTP	Suporte configurável por ambiente	backend/src/services/emailService.js
Zenvia/Twilio	Provedores de SMS	Envio/chechagem opcional de código por SMS	Preparação para validação por telefone	backend/src/services/smsService.js
ViaCEP	API pública de CEP	Buscar endereço por CEP	Facilita cadastro de espaços	backend/src/services/localizacaoService.js
OpenStreet Map/Nominatim	Geocodificação e mapa	Converter endereço em latitude/longitude	Permite mostrar espaços no mapa	backend/src/services/localizacaoService.js , frontend/src/features/map/MapView.jsx
Leaflet	Biblioteca de mapas	Exibir mapas interativos	Leve e compatível com OpenStreetMap	frontend/src/features/map/ , frontend/src/features/owner/LocationPicker.jsx

Tecnologia	O que é	Para que serve	Por que foi utilizada	Onde está no projeto
React Leaflet	Integração React com Leaflet	Criar mapas dentro de componentes React	Evita manipulação manual do mapa	<code>frontend/src/features/map/MapView.jsx</code>
Lucide React	Biblioteca de ícones	Ícones em botões, cards e menus	Melhora clareza visual e consistência	Diversos componentes frontend
Radix Themes	Biblioteca de temas/componentes	Base visual e tema	Ajuda na consistência visual	<code>frontend/src/main.jsx</code>
Playwright	Ferramenta de testes E2E	Testar fluxos no navegador	Valida login, reserva, proprietário e admin	<code>frontend/tests/e2e/</code>
Puppeteer	Automação do Chrome	Geração de PDF/relatórios	Converte HTML/Markdown em PDF	<code>frontend/scripts/generate-report.js</code> e geração deste PDF
ESLint	Análise estática	Encontrar problemas de código	Melhora qualidade e padronização	<code>frontend/eslint.config.js</code>
Server-Sent Events	Comunicação unidirecional servidor -> cliente	Atualizar frontend quando dados mudam	Mais simples que WebSocket para eventos de atualização	<code>backend/src/services/realtimeService.js</code>

4. Estrutura de Pastas

4.1 Estrutura geral

```
playarena v11 final/
├── backend/
│   ├── src/
│   │   ├── app/
│   │   ├── config/
│   │   ├── controllers/
│   │   ├── middleware/
│   │   ├── models/
│   │   ├── routes/
│   │   ├── services/
│   │   ├── utils/
│   │   ├── server.js
│   │   ├── sync-db.js
│   │   ├── check-db.js
│   │   ├── seed-demo.js
│   │   └── backfill-court-coordinates.js
│   ├── uploads/
│   └── package.json
├── frontend/
│   ├── src/
│   │   ├── assets/
│   │   ├── components/
│   │   ├── data/
│   │   ├── features/
│   │   ├── hooks/
│   │   ├── layout/
│   │   ├── services/
│   │   ├── utils/
│   │   ├── App.jsx
│   │   ├── App.css
│   │   ├── index.css
│   │   └── main.jsx
│   ├── tests/
│   ├── scripts/
│   └── package.json
└── docs/
    └── PlayArena_Documentacao_TCC.md
```

Arquivos com sufixo `sync-conflict` encontrados no projeto aparentam ser cópias de conflito de sincronização. Eles não são a base principal importada pela aplicação e devem ser tratados como artefatos residuais, não como fluxo ativo do sistema.

4.2 Backend

backend/src/server.js

É o ponto de entrada do backend. Carrega variáveis de ambiente, cria a aplicação Express, testa a conexão com o banco via Sequelize, expira advertências antigas de proprietários e inicia o servidor na porta `3333` por padrão.

É executado pelos scripts:

- `npm run dev`;
- `npm start`.

backend/src/app/createApp.js

Monta a aplicação Express:

- configura CORS;
- habilita JSON;
- expõe `/uploads`;

- cria `/api/health`;
- registra middleware de transmissão de eventos;
- monta rotas de autenticação, quadras, reservas, admin, notificações, localização e eventos;
- trata 404;
- centraliza tratamento de erro.

backend/src/config/database.js

Configura Sequelize com PostgreSQL. Usa `DATABASE_URL` e SSL, compatível com Neon DB. Também define logs SQL quando `DB_LOGGING=true`.

backend/src/controllers/

Controladores recebem a requisição HTTP, extraem parâmetros e chamam serviços:

- `authController.js` : cadastro, login, perfil e recuperação de senha.
- `quadraController.js` : listagem, criação, edição, exclusão, horários, fotos e documentos.
- `reservaController.js` : criação, listagem, cancelamento, status e validação de reservas.
- `adminController.js` : dashboard, usuários, proprietários, documentos, espaços, administradores e alertas.
- `notificacaoController.js` : listagem e leitura de notificações.
- `localizacaoController.js` : consulta de CEP e geocodificação.

backend/src/routes/

Define os endpoints públicos e protegidos:

- `auth.js` : `/api/auth`.
- `quadras.js` : `/api/quadras`.
- `reservas.js` : `/api/reservas`.
- `admin.js` : `/api/admin`.
- `notificacoes.js` : `/api/notificacoes`.
- `localizacao.js` : `/api/localizacao`.
- `events.js` : `/api/events`.

As rotas chamam controladores e aplicam middlewares como `authenticate`, `requireRoles` e `uploads`.

backend/src/services/

Contém a regra de negócio:

- `authService.js` : validação de cadastro, login, JWT, códigos de verificação, recuperação de senha e perfil.
- `quadraService.js` : regras de criação, edição, listagem, horários e remoção de quadras.
- `reservaService.js` : criação, conflito, cancelamento, reembolso, validação e código de reserva.
- `adminService.js` : gestão administrativa de usuários, proprietários, espaços e dashboard.
- `documentacaoService.js` : criação, reaproveitamento e revisão de documentação de locais.
- `notificacaoService.js` : criação e leitura de notificações.
- `localizacaoService.js` : integração com ViaCEP e Nominatim.
- `emailService.js` : envio de e-mails.
- `smsService.js` : integração opcional com SMS.
- `realtimeService.js` : eventos em tempo real por SSE.
- `userAccessService.js` : cálculo de bloqueios temporários e status efetivo da conta.
- `adminLogService.js` : gravação de ações administrativas.
- `advertenciaProprietarioService.js` : advertências por cancelamento de proprietário e avisos ao administrador.
- `schemaCompatibilityService.js` : ajustes de compatibilidade de schema.

backend/src/models/

Define as tabelas e relacionamentos Sequelize:

- `Usuario.js`;
- `Proprietario.js`;
- `Administrador.js`;
- `Quadra.js`;
- `HorarioDisponivel.js`;
- `Reserva.js`;
- `DocumentacaoLocal.js`;

- `Notificacao.js` ;
- `CadastroPendente.js` ;
- `RecuperacaoSenha.js` ;
- `CredencialBanida.js` ;
- `LogAdministrativo.js` ;
- `AdvertenciaProprietario.js` ;
- `AvisoAdministrativo.js` ;
- `AvisoAdvertenciaItem.js` ;
- `index.js` .

`index.js` inicializa os modelos e monta os relacionamentos.

backend/src/middleware/

- `auth.js` : valida JWT, carrega conta e protege rotas por perfil.
- `profilePhotoUpload.js` : upload de foto de perfil.
- `courtPhotoUpload.js` : upload de fotos de quadras.
- `documentUpload.js` : upload de documentos do proprietário/local.

backend/src/utils/

- `brDocuments.js` : valida CPF e CNPJ.
- `http.js` : utilitários de erro e async handler.

Scripts backend

- `sync-db.js` : sincroniza modelos com o banco.
- `check-db.js` : verifica conexão.
- `seed-demo.js` : cria dados demonstrativos.
- `backfill-court-coordinates.js` : tenta preencher coordenadas de quadras antigas.

4.3 Frontend

frontend/src/main.jsx

Ponto de entrada React. Renderiza o componente `App` dentro do DOM.

frontend/src/App.jsx

Componente raiz. Decide qual portal renderizar de acordo com a sessão:

- sem sessão: `AuthScreen` ;
- perfil `usuario` : `UserPortal` ;
- perfil `proprietario` : `OwnerPortal` ;
- perfil `admin` : `AdminPortal` .

Também renderiza cabeçalho, modais de reserva, sucesso de reserva e toast global.

frontend/src/hooks/usePlayArenaApp.js

É o hook central da aplicação. Controla:

- sessão;
- tela ativa;
- quadras;
- reservas;
- dados do proprietário;
- dados administrativos;
- notificações;
- loading e mensagens;
- login, cadastro, reserva, cancelamento, CRUD de quadras, aprovação, bloqueio e demais ações.

Ele funciona como uma camada de orquestração entre interface e API.

frontend/src/services/

- `httpClient.js` : encapsula `fetch` , monta headers, trata erros, dispara eventos locais e assina SSE.

- `playarenaApi.js` : concentra chamadas aos endpoints do backend.

frontend/src/features/

Organiza telas por domínio:

- `auth/` : login, cadastro e recuperação de senha.
- `courts/` : home, busca, card e detalhe de quadra.
- `reservations/` : modal de reserva, lista de reservas, sucesso e filtros.
- `owner/` : portal do proprietário, formulário de quadra, BI, mapa de localização, card de quadra e reservas.
- `admin/` : portal administrativo, usuários, espaços e avisos.
- `map/` : mapa de quadras.
- `profile/` : perfil e configurações.
- `support/` : suporte.
- `user/` : portal do usuário.

frontend/src/components/

Componentes reutilizáveis:

- `Logo.jsx` ;
- `Metric.jsx` ;
- `NotificationBell.jsx` ;
- `PhotoCarousel.jsx` ;
- `SiteFooter.jsx` ;
- `SupportEmailActions.jsx` ;
- `Toast.jsx` .

frontend/src/data/

- `demoData.js` : dados de apoio, modalidades, dias da semana e imagem fallback.
- `navigation.jsx` : navegação por perfil, incluindo o botão `BI` no dashboard do proprietário.

frontend/src/utils/

- `formatters.js` : formatação de moeda, datas, endereço, CPF/CNPJ e normalização de quadras.
- `sessionStorage.js` : persistência da sessão no navegador.
- `appSettings.js` : idioma, moeda, acessibilidade e tradução simples de textos.

frontend/tests/

Contém testes de unidade e de ponta a ponta:

- `tests/unit/formatters.spec.ts` ;
 - `tests/e2e/auth-and-access.spec.ts` ;
 - `tests/e2e/user-flows.spec.ts` ;
 - `tests/e2e/owner-flows.spec.ts` ;
 - `tests/e2e/admin-flows.spec.ts` ;
 - `tests/e2e/playarena.setup.spec.ts` .
-

5. Explicação das Telas

5.1 Telas comuns e autenticação

Tela	Objetivo	Quem acessa	Componentes/arquivos	APIs usadas
Login e cadastro	Autenticar, cadastrar usuário/proprietário e recuperar senha	Visitantes	frontend/src/features/auth/AuthScreen.jsx	/auth/login , /auth/register , /auth/register/proprietario , /auth/register/confirm , /auth/register/resend , /auth/password/*
Cabeçalho	Navegação, tema, notificações e logout	Usuário autenticado	frontend/src/layout/AppHeader.jsx , NotificationBell.jsx	/notificacoes , /auth/me
Perfil	Atualizar telefone, foto e excluir própria conta	Usuário, proprietário e admin	frontend/src/features/profile/ProfileView.jsx	/auth/profile , /admin/perfil/senha , /auth/profile DELETE
Configurações	Idioma, moeda, acessibilidade e preferências locais	Usuário e proprietário	frontend/src/features/profile/SettingsView.jsx	Predominantemente localStorage
Suporte	Exibir canais de contato	Usuário e proprietário	frontend/src/features/support/SupportView.jsx	Sem API principal

5.2 Telas do usuário

Tela	Objetivo	Quem acessa	Arquivo	APIs usadas
Início	Apresentar busca rápida, esportes e espaços populares	Usuário	frontend/src/features/courts/HomeView.jsx	/quadras
Buscar quadras	Filtrar espaços por texto, modalidade e preço	Usuário	frontend/src/features/courts/SearchView.jsx	/quadras
Detalhe da quadra	Mostrar fotos, preço, endereço, comodidades e horários	Usuário	frontend/src/features/courts/CourtDetail.jsx	/quadras/:id , /quadras/:id/horarios
Reserva	Escolher data, horário, forma de pagamento e confirmar	Usuário	frontend/src/features/reservations/ReservationModal.jsx	/quadras/:id/horarios , /reservas/pre-validar , /reservas
Sucesso da reserva	Exibir código de reserva e opção de contato	Usuário	frontend/src/features/reservations/ReservationSuccess.jsx	Dados retornados de /reservas
Minhas reservas	Acompanhar reservas, código, status e cancelamento	Usuário	frontend/src/features/reservations/ReservationList.jsx	/reservas/minhas , /reservas/:id/cancelar
Mapa	Visualizar quadras geolocalizadas em Campo Mourão	Usuário	frontend/src/features/map/MapView.jsx	/quadras

5.3 Telas do proprietário

Tela	Objetivo	Quem acessa	Arquivo	APIs usadas
Dashboard do proprietário	Mostrar indicadores básicos e atalhos	Proprietário	frontend/src/features/owner/OwnerPortal.jsx	/quadras/minhas , /reservas/proprietario , /quadras/documentacoes/minhas

Tela	Objetivo	Quem acessa	Arquivo	APIs usadas
BI do proprietário	Mostrar dados analíticos, gráficos, rankings e gerar PDF via impressão	Proprietário	frontend/src/features/owner/OwnerBusinessIntelligence.jsx	Usa dados de quadras e reservas já carregados
Meus espaços	Cadastrar e gerenciar quadras	Proprietário	frontend/src/features/owner/OwnerPortal.jsx, CourtForm.jsx, OwnerCourtCard.jsx	/quadras, /quadras/minhas, /quadras/fotos, /quadras/documentos
Cadastro de espaço	Enviar dados, endereço, fotos, horários e documentos	Proprietário	frontend/src/features/owner/CourtForm.jsx	/localizacao/cep/:cep, /localizacao/geocodificar, /quadras/documentos, /quadras/fotos, /quadras
Localização no mapa	Confirmar latitude e longitude do espaço	Proprietário	frontend/src/features/owner/LocationPicker.jsx	/localizacao/geocodificar
Reservas recebidas	Gerenciar status, cancelar e validar código	Proprietário	frontend/src/features/owner/OwnerReservationList.jsx	/reservas/proprietario, /reservas/:id/status, /reservas/:id/cancelar, /reservas/:id/validar-codigo
Perfil do proprietário	Exibir status de aprovação e editar dados	Proprietário	frontend/src/features/owner/OwnerProfile.jsx	/auth/profile

5.4 Telas do administrador

Tela	Objetivo	Quem acessa	Arquivo	APIs usadas
Painel administrativo	Ver indicadores gerais e últimas reservas	Administrador	frontend/src/features/admin/AdminPortal.jsx	/admin/dashboard, /reservas/todas
Documentos dos espaços	Aprovar/reprovar documentação do proprietário/local	Administrador	frontend/src/features/admin/AdminPortal.jsx	/admin/documentacoes, /admin/documentacoes/:id/status
Gestão de espaços	Criar, editar, desativar, reativar, deletar e gerenciar horários	Administrador	frontend/src/features/admin/AdminSpacesView.jsx	/admin/espacos, /quadras/:id/horarios, /quadras/fotos
Reservas do administrador	Acompanhar reservas e atualizar status	Administrador	frontend/src/features/admin/AdminPortal.jsx, OwnerReservationList.jsx	/reservas/todas, /reservas/:id/status, /reservas/:id/cancelar
Usuários e proprietários	Bloquear, banir, desbanir, excluir e aprovar proprietários	Administrador	frontend/src/features/admin/AdminUsersView.jsx	/admin/usuarios, /admin/proprietarios, rotas de bloqueio/banimento/aprovação
Avisos	Monitorar alertas gerados por advertências de proprietários	Administrador	frontend/src/features/admin/AdminAlertsView.jsx	/admin/avisos, /admin/avisos/:id/status
Cadastro de administradores	Criar novos administradores	Super admin	frontend/src/features/admin/AdminPortal.jsx	/admin/administradores
Perfil administrativo	Editar perfil e alterar senha	Administrador	frontend/src/features/profile/ProfileView.jsx	/auth/profile, /admin/perfil/senha

6. Fluxo Completo do Usuário

6.1 Cadastro

Fluxo:

```
Usuário -> AuthScreen -> playarenaApi.registerAccount
-> POST /api/auth/register
-> authController.registerUser
-> authService.registerUser
-> CadastroPendente + e-mail de verificação
```

Arquivos principais:

- frontend/src/features/auth/AuthScreen.jsx ;
- frontend/src/hooks/usePlayArenaApp.js ;
- frontend/src/services/playarenaApi.js ;
- backend/src/routes/auth.js ;
- backend/src/controllers/authController.js ;
- backend/src/services/authService.js ;
- backend/src/models/CadastroPendente.js ;
- backend/src/models/Usuario.js .

Regras:

- exige nome, CPF, e-mail, senha e confirmação;
- valida CPF;
- verifica se credenciais não estão banidas;
- verifica duplicidade de e-mail/CPF;
- cria cadastro pendente com código;
- envia código por e-mail.

6.2 Confirmação de cadastro

Fluxo:

```
Código recebido -> AuthScreen -> POST /api/auth/register/confirm
-> authService.confirmRegistrationCode
-> cria Usuario
```

O código fica armazenado como hash. O sistema limita tentativas e verifica expiração.

6.3 Login

Fluxo:

```
E-mail e senha -> POST /api/auth/login
-> bcrypt.compare
-> valida status da conta
-> gera JWT
-> frontend salva sessão
```

Arquivos:

- AuthScreen.jsx ;
- usePlayArenaApp.js ;
- playarenaApi.js ;
- authService.js ;
- middleware/auth.js .

6.4 Recuperação de senha

Fluxo:

```

Usuário informa e-mail
-> POST /api/auth/password/forgot
-> cria RecuperacaoSenha
-> envia código
-> POST /api/auth/password/verify
-> POST /api/auth/password/reset
-> atualiza senha_hash

```

Arquivos:

- frontend/src/features/auth/AuthScreen.jsx ;
- backend/src/services/authService.js ;
- backend/src/models/RecuperacaoSenha.js ;
- backend/src/services/emailService.js .

6.5 Reserva

Fluxo:

```

Usuário escolhe quadra
-> CourtDetail / ReservationModal
-> GET /api/quadras/:id/horarios
-> usuário seleciona data e horário
-> GET /api/reservas/pre-validar
-> POST /api/reservas
-> reservaService.createReservation
-> Reserva confirmada + código único

```

Arquivos:

- frontend/src/features/courts/CourtDetail.jsx ;
- frontend/src/features/reservations/ReservationModal.jsx ;
- frontend/src/features/reservations/ReservationSuccess.jsx ;
- backend/src/routes/reservas.js ;
- backend/src/services/reservaService.js ;
- backend/src/models/Reserva.js ;
- backend/src/models/HorarioDisponivel.js .

Regras:

- usuário precisa estar autenticado como `usuario` ;
- horário deve existir e estar disponível;
- quadra precisa estar ativa;
- documentação do local precisa estar aprovada;
- não pode haver conflito com outra reserva ativa;
- sistema gera `codigo_reserva` .

6.6 Pagamento

O sistema registra a forma de pagamento escolhida:

- `pix` ;
- `cartao` ;
- `dinheiro` ;
- `nao_informado` .

No estado atual do projeto, não há gateway real de pagamento integrado. A reserva é confirmada pela aplicação e a forma de pagamento fica persistida como informação operacional. A evolução natural é integrar Mercado Pago, Stripe, Asaas ou outro provedor.

6.7 Cancelamento

Fluxo:

```

Minhas Reservas -> Cancelar
-> PATCH /api/reservas/:id/cancelar
-> reservaService.cancelReservation
-> calcula regra de reembolso
-> atualiza status para cancelada

```

Regras principais:

- não cancela reserva concluída ou já cancelada;
- usuário só cancela suas próprias reservas;
- se cancelar com antecedência suficiente, recebe regra de reembolso integral;
- se cancelar em cima da hora, recebe reembolso parcial;
- cancelamentos por proprietário/admin exigem motivo.

6.8 Notificações

Fluxo:

```

Backend cria Notificacao
-> SSE informa data-changed
-> frontend recarrega notificações
-> NotificationBell exibe pendências

```

Arquivos:

- frontend/src/components/NotificationBell.jsx ;
- backend/src/services/notificacaoService.js ;
- backend/src/routes/notificacoes.js ;
- backend/src/models/Notificacao.js ;
- backend/src/services/realtimeService.js .

6.9 Perfil

Fluxo:

```

Perfil -> atualizar telefone/foto
-> PATCH /api/auth/profile
-> authController.updateProfile
-> authService.updateProfile

```

Arquivos:

- frontend/src/features/profile/ProfileView.jsx ;
- backend/src/middleware/profilePhotoUpload.js ;
- backend/src/services/authService.js .

7. Fluxo Completo do Proprietário

7.1 Cadastro

Fluxo:

```
AuthScreen -> cadastro proprietário
-> POST /api/auth/register/proprietario
-> authService.registerOwner
-> CadastroPendente
-> confirmação por e-mail
-> cria Proprietario com status_aprovacao pendente
```

Arquivos:

- frontend/src/features/auth/AuthScreen.jsx ;
- backend/src/services/authService.js ;
- backend/src/models/Proprietario.js ;
- backend/src/models/CadastroPendente.js .

Regras:

- exige nome do responsável, nome da empresa, CPF/CNPJ, e-mail e senha;
- valida CPF ou CNPJ;
- bloqueia credenciais banidas;
- inicia com aprovação pendente.

7.2 Validação documental

Fluxo:

```
Proprietário cadastra espaço
-> envia documentos em CourtForm
-> POST /api/quadras/documentos
-> documentacaoService.ensureCourtDocumentation
-> DocumentacaoLocal pendente
-> admin aprova/reprova
```

Documentos exigidos:

- proprietário dono do local: documento pessoal, CPF, comprovante de endereço e comprovante de posse;
- gerenciador: documento pessoal, CPF, comprovante de endereço e autorização de gerenciamento.

Arquivos:

- frontend/src/features/owner/CourtForm.jsx ;
- backend/src/middleware/documentUpload.js ;
- backend/src/services/documentacaoService.js ;
- backend/src/models/DocumentacaoLocal.js ;
- frontend/src/features/admin/AdminPortal.jsx .

7.3 Criação de espaços

Fluxo:

```
OwnerPortal -> CourtForm
-> consulta CEP/geocodificação
-> upload de fotos/documentos
-> POST /api/quadras
-> quadraService.createCourt
-> cria Quadra + HorarioDisponivel
```

Arquivos:

- `frontend/src/features/owner/CourtForm.jsx` ;
- `frontend/src/features/owner/LocationPicker.jsx` ;
- `backend/src/services/quadraService.js` ;
- `backend/src/services/localizacaoService.js` ;
- `backend/src/models/Quadra.js` ;
- `backend/src/models/HorarioDisponivel.js` .

Regras:

- proprietário precisa estar aprovado;
- endereço precisa ter localização resolvida ou confirmada;
- se a documentação ainda não foi aprovada, o espaço pode ficar inativo até a aprovação.

7.4 Gerenciamento de horários

Fluxo:

```
OwnerCourtCard -> criar/alterar/remover horário
-> POST/PATCH/DELETE /api/quadras/:id/horarios
-> quadraService
-> HorarioDisponivel
```

Tipos:

- horário semanal por dia da semana;
- horário específico por data.

Arquivos:

- `frontend/src/features/owner/OwnerCourtCard.jsx` ;
- `backend/src/services/quadraService.js` ;
- `backend/src/models/HorarioDisponivel.js` .

7.5 Reservas recebidas

Fluxo:

```
OwnerReservationList
-> GET /api/reservas/proprietario
-> lista reservas das quadras do proprietário
```

Arquivos:

- `frontend/src/features/owner/OwnerReservationList.jsx` ;
- `backend/src/services/reservaService.js` .

7.6 Cancelamentos

O proprietário pode cancelar uma reserva informando motivo. O cancelamento:

- atualiza a reserva;
- notifica o usuário;
- pode gerar advertência se acontecer perto do horário da reserva;
- pode gerar aviso administrativo quando o proprietário acumula advertências.

Arquivos:

- `backend/src/services/reservaService.js` ;
- `backend/src/services/advertenciaProprietarioService.js` ;
- `backend/src/models/AdvertenciaProprietario.js` ;
- `backend/src/models/AvisoAdministrativo.js` .

7.7 Promoções

O sistema não possui um CRUD separado de promoções. A funcionalidade atual ocorre quando o preço da quadra é reduzido. O backend identifica a queda de preço e cria notificações de promoção/queda de preço para usuários.

Arquivos:

- `backend/src/services/quadraService.js` ;
- `backend/src/services/notificacaoService.js` ;
- `backend/src/models/Notificacao.js` .

7.8 Notificações

O proprietário recebe notificações de:

- bloqueio;
- advertência;
- avisos relacionados a cancelamentos;
- eventos da plataforma enviados ao perfil proprietário.

7.9 Business Intelligence e relatórios em PDF

O proprietário possui um botão `BI` na área inicial/dashboard. Esse botão abre uma tela de dados com:

- faturamento total;
- reservas totais;
- ticket médio;
- arenas ativas;
- gráfico de faturamento por mês;
- distribuição por status;
- receita por arena;
- receita por modalidade;
- demanda por dia da semana;
- demanda por período do dia;
- ranking de clientes;
- tabela analítica de reservas.

Arquivo principal:

- `frontend/src/features/owner/OwnerBusinessIntelligence.jsx` .

A geração de relatório em PDF é feita por impressão do navegador:

```
OwnerBusinessIntelligence -> botão Exportar PDF -> window.print()
```

Na prática, o navegador permite salvar a visualização como PDF.

8. Fluxo Completo do Administrador

8.1 Login

Fluxo:

```
AuthScreen -> POST /api/auth/login  
-> authService.loginAccount  
-> identifica perfil admin  
-> JWT com perfil admin  
-> AdminPortal
```

Arquivos:

- frontend/src/features/auth/AuthScreen.jsx ;
- frontend/src/features/admin/AdminPortal.jsx ;
- backend/src/services/authService.js ;
- backend/src/models/Administrador.js .

8.2 Validação de proprietários

O administrador pode aprovar ou reprovar proprietários na tela de usuários/proprietários.

Fluxo:

```
AdminUsersView -> onOwnerApproval  
-> PATCH /api/admin/proprietarios/:id/aprovacao  
-> adminService.updateOwnerApproval
```

Arquivos:

- frontend/src/features/admin/AdminUsersView.jsx ;
- backend/src/services/adminService.js ;
- backend/src/models/Proprietario.js .

8.3 Validação documental

Fluxo:

```
AdminPortal/documentos  
-> GET /api/admin/documentacoes  
-> aprovar/reprovar  
-> PATCH /api/admin/documentacoes/:id/status  
-> documentacaoService.reviewDocumentation
```

Ao aprovar documentação, quadras relacionadas podem ser ativadas. Ao reprovar, o administrador precisa informar o motivo.

8.4 Gerenciamento de usuários

O administrador consegue:

- listar usuários e proprietários;
- bloquear temporariamente;
- remover bloqueio;
- banir;
- desbanir;
- excluir permanentemente;
- aprovar/reprovar proprietários.

Arquivos:

- frontend/src/features/admin/AdminUsersView.jsx ;
- backend/src/services/adminService.js ;

- `backend/src/models/CredencialBanida.js` ;
- `backend/src/models/LogAdministrativo.js` .

8.5 Bloqueios

Bloqueios temporários possuem:

- data/hora de início;
- data/hora de fim;
- motivo;
- cálculo de status efetivo.

Arquivos:

- `backend/src/services/userAccessService.js` ;
- `backend/src/services/adminService.js` ;
- `backend/src/middleware/auth.js` .

8.6 Banimentos

Banimento torna a conta inativa. No caso de proprietário, as quadras podem ser desativadas. As credenciais podem ser registradas em `CredencialBanida` para impedir novo cadastro com mesmo e-mail/CPF/CNPJ.

8.7 Notificações

A administração provoca notificações quando aplica bloqueios ou ações que impactam usuários/proprietários.

8.8 Auditoria

Ações administrativas são registradas em `logs_administrativos` por meio de `adminLogService.js` . Isso cria rastreabilidade para decisões sensíveis, como bloqueios, banimentos, exclusões e alterações em espaços.

8.9 Controle geral do sistema

O dashboard administrativo mostra:

- total de usuários;
- total de proprietários;
- total de quadras/espacos;
- total de reservas;
- receita total calculada sobre reservas confirmadas/concluídas;
- últimas reservas.

Arquivos:

- `frontend/src/features/admin/AdminPortal.jsx` ;
 - `backend/src/services/adminService.js` .
-

9. Banco de Dados

9.1 Visão geral

O banco de dados é relacional e modelado com Sequelize. A aplicação usa UUID como chave primária na maioria das tabelas, o que facilita distribuição e evita exposição de IDs sequenciais.

9.2 Tabelas principais

Tabela	Modelo	Chave primária	Função
usuarios	Usuario	id UUID	Contas de usuários finais
proprietarios	Proprietario	id UUID	Contas de donos/gerenciadores de espaços
administradores	Administrador	id UUID	Contas administrativas
quadras	Quadra	id UUID	Espaços esportivos cadastrados
horarios_disponiveis	HorarioDisponivel	id UUID	Horários semanais ou por data
reservas	Reserva	id UUID	Reservas realizadas
documentacoes_locais	DocumentacaoLocal	id UUID	Documentos vinculados a um endereço/local
notificacoes	Notificacao	id UUID	Notificações de usuários/proprietários
cadastros_pendentes	CadastroPendente	id UUID	Cadastros aguardando confirmação
recuperacoes_senha	RecuperacaoSenha	id UUID	Fluxo de recuperação de senha
credenciais_banidas	CredencialBanida	id UUID	E-mails/CPFs/CNPJs bloqueados para novo cadastro
logs_administrativos	LogAdministrativo	id UUID	Auditoria administrativa
advertencias_proprietarios	AdvertenciaProprietario	id UUID	Advertências por cancelamento do proprietário
avisos_administrativos	AvisoAdministrativo	id UUID	Alertas gerados para o administrador
avisos_advertencias_itens	AvisoAdvertenciaItem	id UUID	Tabela de associação entre avisos e advertências

9.3 Relacionamentos principais

```

Proprietario 1:N Quadra
Proprietario 1:N DocumentacaoLocal
Proprietario 1:N AdvertenciaProprietario
Proprietario 1:N AvisoAdministrativo

DocumentacaoLocal 1:N Quadra

Quadra 1:N HorarioDisponivel
Quadra 1:N Reserva
Quadra 1:N Notificacao
Quadra 1:N AdvertenciaProprietario

Usuario 1:N Reserva
Usuario 1:N Notificacao

HorarioDisponivel 1:N Reserva

Reserva 1:N Notificacao
Reserva 1:1 AdvertenciaProprietario

Administrador 1:N LogAdministrativo
Administrador 1:N DocumentacaoLocal analisada

AvisoAdministrativo N:N AdvertenciaProprietario
por meio de AvisoAdvertenciaItem
  
```

9.4 MER textual

```

[USUARIO]
id PK
nome
cpf UNIQUE
email UNIQUE
senha_hash
telefone
status
foto_perfil_url
bloqueio_inicio/bloqueio_fim/bloqueio_motivo

[PROPRIETARIO]
id PK
nome_responsavel
nome_empresa
cpf_cnpj UNIQUE
email UNIQUE
senha_hash
telefone
status_aprovacao
ativo
bloqueio_inicio/bloqueio_fim/bloqueio_motivo

[ADMINISTRADOR]
id PK
nome
cpf UNIQUE
email UNIQUE
senha_hash
nivel_acesso
ativo

[DOCUMENTACAO_LOCAL]
id PK
proprietario_id FK -> PROPRIETARIO.id
administrador_id FK -> ADMINISTRADOR.id
endereco_key
endereco/bairro/cidade/estado/cep
documentos JSONB
status
motivo_reprovacao

[QUADRA]
id PK
proprietario_id FK -> PROPRIETARIO.id
documentacao_local_id FK -> DOCUMENTACAO_LOCAL.id
nome
modalidade
modalidades JSONB
tipo_espaco
endereco/bairro/cidade/estado/cep/numero
latitude/longitude
preco_hora
preco_original
fotos JSONB
amenities JSONB
ativa
desativada_inicio/desativada_fim

[HORARIO_DISPONIVEL]
id PK
quadra_id FK -> QUADRA.id
data
dia_semana
hora_inicio

```



```

hora_fim
disponivel
valor
valor_especial

[RESERVA]
id PK
usuario_id FK -> USUARIO.id
quadra_id FK -> QUADRA.id
horario_disponivel_id FK -> HORARIO_DISPONIVEL.id
data_reserva
hora_inicio
hora_fim
status
forma_pagamento
valor_total
codigo_reserva UNIQUE
status_validacao
validado_por_id FK -> PROPRIETARIO.id
cancelada_por
motivo_cancelamento

[NOTIFICACAO]
id PK
userId FK -> USUARIO.id
recipientId
recipientPerfil
reservationId FK -> RESERVA.id
quadraId FK -> QUADRA.id
title
message
type
isRead
metadata JSONB

```

9.5 Índices e regras relevantes

- Usuários: e-mail e CPF únicos.
- Proprietários: e-mail e CPF/CNPJ únicos.
- Administradores: e-mail e CPF únicos.
- Reservas: código de reserva único.
- Horários: índice único por quadra, data/dia da semana e intervalo.
- Reservas ativas: controle para evitar conflito no mesmo espaço, data e horário.
- Documentação: combinação de proprietário e chave de endereço evita duplicação do mesmo local.
- Bloqueios: índices por intervalo de bloqueio ajudam a consultar status temporário.
- Notificações: índices por destinatário, leitura e data.

9.6 Regras de negócio no banco e serviços

- A conta não armazena senha em texto puro, apenas `senha_hash`.
- Cadastro confirmado só é criado após código válido.
- Quadra pública depende de documentação aprovada e status ativo.
- Reserva não pode duplicar horário ativo.
- Reserva concluída não pode ser cancelada.
- Proprietário que cancela em cima da hora pode receber advertência.
- Três advertências ativas geram aviso administrativo.
- Exclusão administrativa pode registrar credencial banida para impedir recadastro.

10. APIs

10.1 Autenticação

Método e rota	O que faz	Controller	Serviço	Modelos principais	Retorno esperado
POST /api/auth/register	Inicia cadastro de usuário	authController.registerUser	authService.registerUser	CadastroPendente , Usuario	ID de verificação e mensagem
POST /api/auth/register/proprietario	Inicia cadastro de proprietário	authController.registerOwner	authService.registerOwner	CadastroPendente , Proprietario	ID de verificação e mensagem
POST /api/auth/register/confirm	Confirma código de cadastro	authController.confirmRegistration	authService.confirmRegistrationCode	CadastroPendente , Usuario , Proprietario	Conta criada
POST /api/auth/register/resend	Reenvia código de cadastro	authController.resendRegistrationCode	authService.resendRegistrationCode	CadastroPendente	Mensagem de reenvio
POST /api/auth/login	Autentica conta	authController.login	authService.loginAccount	Usuario , Proprietario , Administrador	JWT, perfil e conta
POST /api/auth/password/forgot	Solicita recuperação de senha	authController.requestPasswordReset	authService.requestPasswordReset	RecuperacaoSenha	ID de reset
POST /api/auth/password/verify	Valida código de recuperação	authController.verifyPasswordResetCode	authService.verifyPasswordResetCode	RecuperacaoSenha	Código validado
POST /api/auth/password/reset	Altera senha	authController.resetPassword	authService.resetPassword	Usuario , Proprietario	Mensagem de sucesso
GET /api/auth/me	Retorna conta atual	authController.me	Middleware auth	Conta autenticada	Perfil sanitizado
PATCH /api/auth/profile	Atualiza perfil/foto	authController.updateProfile	authService.updateProfile	Conta autenticada	Conta atualizada
DELETE /api/auth/profile	Exclui própria conta	authController.deleteProfileAccount	authService.deleteAccount	Usuario ou Proprietario	Mensagem de exclusão

10.2 Quadras e espaços

Método e rota	O que faz	Controller	Serviço	Modelos principais	Retorno esperado
GET /api/quadras	Lista quadras públicas	quadraController.listCourts	quadraService.listCourts	Quadra , Proprietario	Lista de quadras
GET /api/quadras/minhas	Lista quadras do proprietário	quadraController.listOwnerCourts	quadraService.listOwnerCourts	Quadra	Lista do proprietário
GET /api/quadras/documentacoes/minhas	Lista documentações do proprietário	quadraController.listOwnerDocumentations	documentacaoService.listOwnerDocumentations	DocumentacaoLocal	Lista de documentações
POST /api/quadras/documentos	Faz upload de documentos	quadraController.uploadDocuments	Middleware upload	Arquivos locais	URLs dos documentos

Método e rota	O que faz	Controller	Serviço	Modelos principais	Retorno esperado
POST /api/quadras/fotos	Faz upload de fotos	quadraController.uploadPhotos	Middleware upload	Arquivos locais	URLs das fotos
GET /api/quadras/:id	Detalha quadra pública	quadraController.getCourt	quadraService.getCourt	Quadra	Dados da quadra
POST /api/quadras	Cria quadra	quadraController.createCourt	quadraService.createCourt	Quadra , HorárioDisponível , DocumentacaoLocal	Quadra(s) criada(s)
PUT /api/quadras/:id	Atualiza quadra	quadraController.updateCourt	quadraService.updateCourt	Quadra	Quadra atualizada
DELETE /api/quadras/:id	Remove quadra	quadraController.deleteCourt	quadraService.deleteCourtPermanently	Quadra	Mensagem de exclusão
GET /api/quadras/:id/horarios	Lista horários	quadraController.listSchedules	quadraService.listSchedules	HorárioDisponível , Reserva	Horários disponíveis
POST /api/quadras/:id/horarios	Cria horário	quadraController.createSchedule	quadraService.createSchedule	HorárioDisponível	Horário criado
PATCH /api/quadras/:quadraId/horarios/:horarioId/disponibilidade	Altera disponibilidade	quadraController.updateScheduleAvailability	quadraService.updateScheduleAvailability	HorárioDisponível	Horário atualizado
DELETE /api/quadras/:quadraId/horarios/:horarioId	Exclui horário	quadraController.deleteSchedule	quadraService.deleteSchedule	HorárioDisponível	Mensagem de exclusão

10.3 Reservas

Método e rota	O que faz	Controller	Serviço	Modelos principais	Retorno esperado
POST /api/reservas	Cria reserva	reservaController.createReservation	reservaService.createReservation	Reserva , Quadra , HorárioDisponível	Reserva confirmada
GET /api/reservas/pre-validar	Informa risco de reserva em cima da hora	reservaController.previewReservationTiming	reservaService.previewReservationTiming	Sem gravação	Aviso de reembolso
GET /api/reservas/minhas	Lista reservas do usuário	reservaController.listMyReservations	reservaService.listUserReservations	Reserva	Lista de reservas
GET /api/reservas/proprietario	Lista reservas recebidas	reservaController.listOwnerReservations	reservaService.listOwnerReservations	Reserva , Quadra	Lista do proprietário
GET /api/reservas/todas	Lista reservas para admin	reservaController.listAllReservations	reservaService.listAllReservations	Reserva	Lista geral
PATCH /api/reservas/:id/status	Atualiza status	reservaController.updateStatus	reservaService.updateReservationStatus	Reserva	Reserva atualizada
PATCH /api/reservas/:id/validar-codigo	Valida presença/código	reservaController.validateReservationCode	reservaService.validateReservationCode	Reserva	Reserva concluída

Método e rota	O que faz	Controller	Serviço	Modelos principais	Retorno esperado
PATCH /api/reservas/:id/cancelar	Cancela reserva	reservaController.cancelReservation	reservaService.cancelReservation	Reserva, Notificacao, AdvertenciaProprietario	Reserva cancelada

10.4 Administração

Método e rota	O que faz	Controller	Serviço	Modelos principais	Retorno esperado
GET /api/admin/dashboard	Indicadores gerais	adminController.dashboard	adminService.getDashboard	Vários	Indicadores
GET /api/admin/usuarios	Lista usuários	adminController.listUsers	adminService.listUsers	Usuario	Lista
PATCH /api/admin/usuarios/:id/bloqueio-temporario	Bloqueia usuário	adminController.blockUserTemporarily	adminService.blockUserTemporarily	Usuario, LogAdministrativo	Usuário atualizado
DELETE /api/admin/usuarios/:id/bloqueio-temporario	Remove bloqueio	adminController.clearUserTemporaryBlock	adminService.clearUserTemporaryBlock	Usuario	Usuário atualizado
DELETE /api/admin/usuarios/:id	Bane usuário	adminController.banUser	adminService.banUser	Usuario	Usuário banido
PATCH /api/admin/usuarios/:id/desbanir	Desbane usuário	adminController.unbanUser	adminService.unbanUser	Usuario	Usuário reativado
DELETE /api/admin/usuarios/:id/deletar-conta	Exclui usuário	adminController.deleteUserAccount	adminService.deleteUserAccount	Usuario, CredencialBandida	Mensagem
GET /api/admin/proprietarios	Lista proprietários	adminController.listOwners	adminService.listOwners	Proprietario	Lista
PATCH /api/admin/proprietarios/:id/bloqueio-temporario	Bloqueia proprietário	adminController.blockOwnerTemporarily	adminService.blockOwnerTemporarily	Proprietario	Proprietário atualizado
DELETE /api/admin/proprietarios/:id/bloqueio-temporario	Remove bloqueio	adminController.clearOwnerTemporaryBlock	adminService.clearOwnerTemporaryBlock	Proprietario	Proprietário atualizado
DELETE /api/admin/proprietarios/:id	Bane proprietário	adminController.banOwner	adminService.banOwner	Proprietario, Quadra	Proprietário banido
PATCH /api/admin/proprietarios/:id/desbanir	Desbane proprietário	adminController.unbanOwner	adminService.unbanOwner	Proprietario	Proprietário reativado
DELETE /api/admin/proprietarios/:id/deletar-conta	Exclui proprietário	adminController.deleteOwnerAccount	adminService.deleteOwnerAccount	Proprietario, CredencialBandida	Mensagem
PATCH /api/admin/proprietarios/:id/aprovacao	Aprova/reprova proprietário	adminController.updateOwnerApproval	adminService.updateOwnerApproval	Proprietario	Proprietário atualizado
GET /api/admin/documentacoes	Lista documentações	adminController.listDocumentations	documentacaoService.listDocumentations	DocumentacaoLocal	Lista
PATCH /api/admin/documentacoes/:id/status	Revisa documentação	adminController.reviewDocumentation	documentacaoService.reviewDocumentation	DocumentacaoLocal, Quadra	Documentação atualizada

Método e rota	O que faz	Controller	Serviço	Modelos principais	Retorno esperado
GET /api/admin/avisos	Lista avisos administrativos	adminController.listAdminAlerts	advertenciaProprietarioService.listAvisosAdministrativos	AvisoAdministrativo	Lista
GET /api/admin/avisos/:id	Detalha aviso	adminController.getAdminAlert	advertenciaProprietarioService.getAvisoAdministrativo	AvisoAdministrativo	Aviso
PATCH /api/admin/avisos/:id/status	Atualiza status do aviso	adminController.updateAdminAlertStatus	advertenciaProprietarioService.updateAvisoStatus	AvisoAdministrativo	Aviso atualizado
GET /api/admin/administradores	Lista admins	adminController.listAdmins	adminService.listAdmins	Administrador	Lista
POST /api/admin/administradores	Cria admin	adminController.createAdmin	adminService.createAdmin	Administrador	Admin criado
PATCH /api/admin/perfil/senha	Troca senha admin	adminController.changeOwnPassword	adminService.changeOwnPassword	Administrador	Mensagem
GET /api/admin/espacos	Lista espaços	adminController.listSpaces	adminService.listSpaces	Quadra	Lista
POST /api/admin/espacos	Cria espaço como admin	adminController.createSpace	adminService.createSpace	Quadra	Espaço criado
PUT /api/admin/espacos/:id	Atualiza espaço	adminController.updateSpace	adminService.updateSpace	Quadra	Espaço atualizado
PATCH /api/admin/espacos/:id/desativacao	Agenda/desativa espaço	adminController.deactivateSpace	adminService.deactivateSpace	Quadra	Espaço atualizado
DELETE /api/admin/espacos/:id/desativacao	Limpa desativação	adminController.clearSpaceDeactivation	adminService.clearSpaceDeactivation	Quadra	Espaço reativado
DELETE /api/admin/espacos/:id	Deleta espaço	adminController.deleteSpace	adminService.deleteSpace	Quadra	Mensagem

10.5 Notificações, localização e eventos

Método e rota	O que faz	Controller/serviço	Retorno esperado
GET /api/notificacoes	Lista notificações do usuário/proprietário	notificacaoController.listNotifications	Lista e contagem não lida
PATCH /api/notificacoes/read-all	Marca todas como lidas	notificacaoController.markAllAsRead	Contagem atualizada
PATCH /api/notificacoes/:id/read	Marca uma como lida	notificacaoController.markAsRead	Notificação atualizada
GET /api/localizacao/cep/:cep	Consulta CEP	localizacaoController.lookupCep	Endereço
GET /api/localizacao/geocodificar	Geocodifica endereço	localizacaoController.geocodeAddress	Latitude/longitude
GET /api/events	Abre canal SSE	realtimeService.addClient	Eventos em tempo real
GET /api/health	Verifica saúde da API	createApp.js	Status ok

11. Segurança

11.1 JWT

O JWT autentica requisições sem manter sessão no servidor. O backend assina o token com `JWT_SECRET` e define expiração de 7 dias.

Vantagens:

- reduz estado no servidor;
- facilita proteção de rotas;
- permite identificar perfil da conta rapidamente.

Arquivo:

- `backend/src/middleware/auth.js`.

11.2 Middleware de autenticação

O middleware:

- extrai o token do cabeçalho `Authorization`;
- valida assinatura e expiração;
- carrega a conta no banco;
- verifica status ativo/inativo;
- verifica bloqueio temporário;
- injeta `request.auth`.

Rotas usam:

```
authenticate
requireRoles('usuario')
requireRoles('proprietario')
requireRoles('admin')
```

11.3 Validação de CPF e CNPJ

A validação fica em `backend/src/utils/brDocuments.js`. O código remove caracteres não numéricos, rejeita sequências repetidas e calcula dígitos verificadores.

Usos:

- cadastro de usuário;
- cadastro de proprietário;
- cadastro de administrador;
- bloqueio de credenciais duplicadas ou banidas.

11.4 Hash de senhas e códigos

Senhas e códigos são protegidos com `bcrypt`. Isso impede que uma exposição de banco revele senhas diretamente.

Usos:

- `senha_hash` em usuários, proprietários e administradores;
- `codigo_hash` em cadastros pendentes;
- `codigo_hash` em recuperação de senha.

11.5 Upload seguro

Os middlewares de upload:

- limitam tamanho do arquivo;
- restringem tipos permitidos;
- salvam com nomes únicos;
- separam fotos de perfil, fotos de quadra e documentos;
- bloqueiam duplicidade de documentos em campos diferentes.

Limites observados:

- documentos: até 8 MB por arquivo;
- fotos de quadra: até 5 MB por arquivo;
- foto de perfil: até 5 MB.

11.6 Proteção de rotas

Exemplos:

- só usuário cria reserva;
- só proprietário lista suas reservas recebidas;
- só proprietário/admin cria horário;
- só admin acessa `/api/admin` ;
- só super admin visualiza/cria administradores.

11.7 CORS

O backend aceita origens locais como `localhost` e `127.0.0.1` , além de `FRONTEND_URL` . Isso protege contra origens não autorizadas em produção.

11.8 Auditoria

Operações sensíveis de admin são gravadas em `logs_administrativos` . Isso é importante para rastreabilidade, prestação de contas e investigação de abuso.

12. Funcionalidades Implementadas

12.1 Usuário

- Cadastro com verificação por e-mail.
- Login com JWT.
- Recuperação de senha.
- Busca de quadras.
- Filtro por modalidade e preço.
- Visualização de detalhes da quadra.
- Visualização de fotos.
- Visualização de comodidades.
- Visualização de horários.
- Reserva de horário.
- Escolha de forma de pagamento informativa.
- Código de reserva.
- Histórico de reservas.
- Cancelamento de reserva.
- Regras de reembolso.
- Mapa de quadras.
- Notificações.
- Perfil com foto e telefone.
- Exclusão da própria conta.
- Configurações de idioma, moeda e acessibilidade.

12.2 Proprietário

- Cadastro com CPF/CNPJ.
- Confirmação por e-mail.
- Status de aprovação.
- Envio de documentação.
- Cadastro de quadras/espacos.
- Cadastro de múltiplas unidades em lote.
- Upload de fotos.
- Consulta de CEP.
- Geocodificação de endereço.
- Confirmação de localização no mapa.
- Cadastro de horários fixos e específicos.
- Gerenciamento de horários.
- Edição de preço, fotos e comodidades.
- Exclusão de espaço.
- Listagem de reservas recebidas.
- Alteração de status de reserva.
- Cancelamento com motivo.
- Validação de código de reserva.
- Advertências por cancelamento em cima da hora.
- BI com gráficos, indicadores e tabelas.
- Relatório PDF por impressão do navegador.

12.3 Administrador

- Login administrativo.
- Dashboard geral.
- Listagem de usuários.
- Listagem de proprietários.
- Aprovação/reprovação de proprietários.
- Bloqueio temporário.
- Remoção de bloqueio.

- Banimento.
- Desbanimento.
- Exclusão permanente.
- Registro de credenciais banidas.
- Listagem de documentações.
- Aprovação/reprovação documental.
- Gestão de espaços.
- Criação de espaços.
- Edição de espaços.
- Desativação temporária/agendada.
- Reativação.
- Deleção.
- Gestão de horários.
- Listagem geral de reservas.
- Gestão de avisos administrativos.
- Cadastro de administradores por super admin.
- Alteração de senha.
- Logs administrativos.

12.4 Plataforma

- API REST.
 - PostgreSQL com Sequelize.
 - Upload local.
 - Notificações persistidas.
 - Eventos em tempo real com SSE.
 - Integração ViaCEP.
 - Integração Nominatim/OpenStreetMap.
 - Testes com Playwright.
 - Geração de relatórios com Puppeteer.
-

13. Funcionalidades Futuras

1. Integração real de pagamento com gateway.
 2. Confirmação automática de pagamento.
 3. Estorno financeiro integrado.
 4. Avaliação de quadras por usuários.
 5. Chat interno entre usuário e proprietário.
 6. Calendário visual para proprietários.
 7. Controle de planos/assinaturas para proprietários.
 8. App mobile com React Native.
 9. Painel público de rankings de quadras.
 10. Cupons de desconto.
 11. Promoções com data de início/fim.
 12. Favoritos.
 13. Histórico financeiro detalhado.
 14. Exportação de relatórios em Excel/CSV.
 15. Armazenamento de arquivos em S3/Cloudflare R2.
 16. Autenticação em dois fatores.
 17. Login social com Google.
 18. Integração com WhatsApp Business API.
 19. Expansão para outras cidades.
 20. Algoritmo de recomendação por esporte e distância.
 21. Política de cancelamento configurável por proprietário.
 22. Dashboard financeiro do administrador.
 23. Módulo fiscal/nota.
 24. Observabilidade com logs estruturados e métricas.
 25. Migrações versionadas com Sequelize CLI.
-

14. Perguntas Possíveis da Banca

Nº	Pergunta	Resposta curta	Resposta técnica	Resposta completa
1	O que é o PlayArena?	Uma plataforma de reservas esportivas.	SPA React com API Express e PostgreSQL.	É um sistema que conecta usuários que querem reservar quadras a proprietários que gerenciam espaços esportivos, com apoio administrativo.
2	Qual problema o sistema resolve?	Resolve a desorganização de reservas.	Centraliza disponibilidade, reserva, documentação e gestão.	Ele substitui processos manuais por um fluxo digital, reduz conflito de horários e melhora controle para usuários, proprietários e admin.
3	Por que escolher React?	Para criar interface dinâmica.	React facilita componentes reutilizáveis e estado reativo.	Como o sistema tem muitos painéis, filtros, modais e atualizações, React ajuda a organizar a interface em componentes.
4	Por que usar Vite?	Porque é rápido.	Vite fornece dev server e build otimizado.	Ele acelera desenvolvimento frontend e simplifica configuração de projetos React modernos.
5	Por que usar Node.js?	Para criar o backend em JavaScript.	Node executa Express e integra bibliotecas como Sequelize e JWT.	A equipe pode usar JavaScript no frontend e backend, reduzindo mudança de contexto.
6	Qual é o papel do Express?	Criar a API.	Define rotas, middlewares e tratamento HTTP.	O Express recebe requisições do frontend, aplica autenticação e chama controladores/serviços.
7	Por que PostgreSQL?	Porque é relacional e robusto.	Suporta chaves estrangeiras, índices, transações e JSONB.	O domínio exige relacionamentos entre usuários, quadras, horários e reservas; PostgreSQL atende bem isso.
8	O que é Sequelize?	Um ORM.	Mapeia modelos JavaScript para tabelas SQL.	Ele facilita consultas, relacionamentos e operações no banco sem escrever SQL manual em toda regra.
9	Como funciona o login?	E-mail e senha geram JWT.	O backend compara bcrypt, valida status e assina token.	O frontend envia credenciais, o backend valida conta e retorna token usado em rotas protegidas.
10	Como a senha é protegida?	Com hash bcrypt.	A senha vira <code>senha_hash</code> e nunca é salva pura.	Mesmo se o banco for exposto, a senha original não aparece diretamente.
11	O que é JWT?	Um token de autenticação.	Token assinado com <code>sub</code> , <code>perfil</code> e expiração.	Ele identifica a conta em cada requisição sem depender de sessão em memória no servidor.
12	Como as rotas são protegidas?	Por middleware.	<code>authenticate</code> valida token e <code>requireRoles</code> valida perfil.	Cada rota sensível exige perfil correto, como usuário para reservar e admin para gestão.
13	Como o cadastro é confirmado?	Por código enviado por e-mail.	<code>CadastroPendente</code> guarda payload e hash do código.	O cadastro só vira conta real após confirmar o código dentro do prazo.
14	O sistema valida CPF/CNPJ?	Sim.	Usa cálculo de dígitos verificadores.	Isso evita documentos inválidos e reduz cadastros falsos.
15	Como uma quadra é cadastrada?	Pelo proprietário ou admin.	Cria <code>Quadra</code> , documentação e horários.	O proprietário informa dados, endereço, fotos, documentos e horários; o backend valida e persiste.
16	Uma quadra já aparece pública ao cadastrar?	Depende da documentação.	A listagem pública filtra quadras ativas e aprovadas.	Espaços com documentação pendente não devem aparecer como disponíveis ao usuário final.
17	Como funciona a validação documental?	Admin aprova ou reprova.	<code>DocumentacaoLocal</code> possui status e motivo de reprovação.	O proprietário envia documentos, o administrador analisa e a decisão afeta a ativação do espaço.
18	Como o sistema evita conflito de reservas?	Valida horário antes de reservar.	<code>reservaService</code> consulta reservas ativas para mesma quadra/data/hora.	Antes de criar reserva, o backend verifica se o intervalo já não está ocupado.
19	Como o código de reserva é usado?	Para validar presença.	<code>codigo_reserva</code> é único e conferido pelo proprietário.	O usuário apresenta o código, o proprietário valida e a reserva é marcada como concluída.
20	O sistema tem pagamento real?	Ainda não.	Ele persiste apenas a forma de pagamento escolhida.	Pix/cartão/dinheiro são registrados, mas não há gateway financeiro integrado no estado atual.

Nº	Pergunta	Resposta curta	Resposta técnica	Resposta completa
21	Como funciona o cancelamento?	Atualiza a reserva para cancelada.	<code>reservaService.cancelReservation</code> aplica regras de permissão e reembolso.	Usuário, proprietário ou admin podem cancelar conforme vínculo e regras; proprietário/admin precisam informar motivo.
22	O que acontece se proprietário cancelar em cima da hora?	Pode receber advertência.	<code>advertenciaProprietarioService</code> cria advertência ativa.	Advertências acumuladas geram aviso administrativo para monitoramento da plataforma.
23	O que é o BI do proprietário?	Tela de indicadores.	Calcula métricas a partir de quadras e reservas do proprietário.	Mostra faturamento, reservas, ticket médio, demanda por período, ranking e gráficos.
24	Como gera PDF no BI?	Pela impressão do navegador.	O componente chama <code>window.print()</code> .	O usuário clica em exportar e escolhe salvar como PDF na janela de impressão.
25	O sistema usa Socket.io?	Não.	Usa Server-Sent Events.	Para atualizações simples servidor-cliente, SSE atende com menor complexidade.
26	O que são SSE?	Eventos enviados pelo servidor.	Conexão HTTP mantida em <code>/api/events</code> .	O backend avisa o frontend quando dados mudam, e o frontend recarrega informações relevantes.
27	Como as notificações funcionam?	São salvas no banco e exibidas no sino.	<code>Notificacao</code> guarda destinatário, tipo, leitura e metadados.	Serviços criam notificações em ações importantes, e o frontend lista/marca como lida.
28	Como o mapa funciona?	Com Leaflet e OpenStreetMap.	Quadras usam latitude/longitude.	O backend geocodifica endereços e o frontend exibe marcadores em mapa interativo.
29	Por que usar ViaCEP?	Para preencher endereço.	Consulta CEP no cadastro de espaço.	Melhora usabilidade e reduz erro de digitação no endereço.
30	Qual a função do Nominatim?	Geocodificar endereço.	Retorna coordenadas para o mapa.	A partir de endereço, cidade e CEP, o sistema obtém latitude e longitude.
31	Onde ficam os arquivos enviados?	Em <code>backend/uploads</code> .	Multer grava documentos e imagens localmente.	O banco guarda URLs/caminhos, e o Express serve os arquivos.
32	O upload é seguro?	Possui restrições básicas.	Limita tamanho, MIME type e campos aceitos.	Isso reduz risco de arquivos indevidos, embora produção ideal use antivírus/storage externo.
33	Como funciona o administrador?	Gerencia a plataforma.	Usa rotas <code>/api/admin</code> protegidas por perfil admin.	Admin valida documentos, usuários, espaços, reservas, bloqueios e alertas.
34	O que é super admin?	Admin com permissão maior.	<code>nivel_acesso = super_admin</code> .	Apenas super admin pode listar/criar administradores.
35	Como bloquear usuário temporariamente?	Admin define início, fim e motivo.	Campos de bloqueio são checados no middleware.	Durante o período, o login/aceso pode ser impedido e o usuário recebe notificação.
36	O que é banimento?	Desativação permanente da conta.	Status/ativo é alterado e credenciais podem ser banidas.	Isso impede acesso e pode bloquear recadastro com as mesmas credenciais.
37	Como existe auditoria?	Com logs administrativos.	<code>LogAdministrativo</code> registra ação, entidade e detalhes.	A plataforma sabe qual admin executou ações sensíveis e quando.
38	Quais perfis existem?	Usuário, proprietário e admin.	O JWT traz o campo <code>perfil</code> .	A interface e o backend usam o perfil para liberar funcionalidades específicas.
39	Como o frontend sabe qual portal abrir?	Pela sessão.	<code>App.jsx</code> verifica <code>session.perfil</code> .	Após login, o token e dados da conta são armazenados e o portal correto é renderizado.
40	Qual arquivo central do frontend?	<code>usePlayArenaApp.js</code> .	Ele concentra estado e handlers.	Esse hook orquestra chamadas de API e atualizações de interface.
41	Qual arquivo central do backend?	<code>server.js</code> e <code>createApp.js</code> .	Um inicia servidor, outro monta Express.	Juntos configuram API, middlewares, rotas e tratamento de erro.

Nº	Pergunta	Resposta curta	Resposta técnica	Resposta completa
42	Como os testes são feitos?	Com Playwright e testes unitários.	E2E cobre fluxos de usuário, proprietário, admin e autenticação.	Playwright automatiza o navegador para validar cenários reais da aplicação.
43	O que é Neon DB?	PostgreSQL em nuvem.	O projeto usa <code>DATABASE_URL</code> com SSL.	Neon permite hospedar o banco PostgreSQL sem infraestrutura própria.
44	Por que usar UUID?	Evita IDs sequenciais.	Modelos usam UUID como PK.	Isso melhora segurança por não expor contagem simples e facilita integração distribuída.
45	Como o sistema lida com idioma/moeda?	Configurações locais.	<code>appSettings.js</code> aplica traduções e conversões.	O usuário pode ajustar idioma, moeda e acessibilidade no frontend.
46	O que é <code>DocumentacaoLocal</code> ?	Documentos por endereço/local.	Relaciona proprietário, endereço e status.	Permite validar um local e associar várias quadras ao mesmo endereço.
47	Como quadras são listadas publicamente?	Apenas ativas/aprovadas.	<code>quadraService.listCourts</code> aplica filtros.	Usuário final só deve ver espaços aptos para reserva.
48	Como horários podem ser cadastrados?	Por dia fixo ou data específica.	<code>HorarioDisponivel</code> tem <code>dia_semana</code> ou <code>data</code> .	Isso permite agenda semanal recorrente ou horários pontuais.
49	Qual é o maior risco técnico atual?	Pagamento ainda não integrado.	Forma de pagamento é informativa.	Para produção comercial, seria necessário integrar gateway e conciliação financeira.
50	Como o sistema pode escalar?	Separando storage, banco e API.	Usar storage externo, cache, filas e observabilidade.	A base atual permite evoluir para produção com S3/R2, gateway, logs estruturados e deploy cloud.
51	O que acontece se o backend cair?	Frontend perde chamadas de API.	<code>httpClient</code> retorna erro tratado.	O usuário verá mensagens de erro; a persistência depende do backend e banco ativos.
52	O que garante integridade nas reservas?	Regra no serviço e índices.	Serviço valida conflito e banco possui índices.	A aplicação combina validação de negócio e estrutura de banco para reduzir duplicidades.
53	Onde as regras de negócio devem ficar?	No backend.	Services centralizam regras.	O frontend ajuda na experiência, mas a validação crítica precisa estar no servidor.
54	Por que separar controller e service?	Organização.	Controller trata HTTP; service trata negócio.	Essa separação facilita manutenção, teste e evolução da API.
55	Qual melhoria de segurança você faria?	Storage externo e hardening.	Adicionar rate limit, helmet, antivírus e refresh tokens.	A base é boa para TCC, mas produção exige camadas extras contra abuso.

15. Roteiro de Apresentação de 15 Minutos

0:00 - 1:30 Introdução

Apresentar o projeto:

- nome PlayArena;
- cidade inicial Campo Mourão - PR;
- objetivo de conectar usuários e proprietários de espaços esportivos.

Frase sugerida:

O PlayArena é uma plataforma web para facilitar a reserva e o gerenciamento de quadras esportivas, atendendo usuários, proprietários e administradores.

1:30 - 3:00 Problema

Explicar:

- reservas feitas manualmente;
- dificuldade de saber disponibilidade;
- conflitos de horário;
- falta de controle para proprietários;
- falta de validação de espaços.

3:00 - 4:30 Solução

Mostrar que o PlayArena centraliza:

- busca;
- reserva;
- cadastro de espaços;
- validação documental;
- gerenciamento administrativo;
- notificações;
- indicadores.

4:30 - 6:00 Tecnologias

Apresentar rapidamente:

- React + Vite no frontend;
- Node.js + Express no backend;
- Sequelize + PostgreSQL/Neon no banco;
- JWT + bcrypt na segurança;
- Multer para uploads;
- ViaCEP e OpenStreetMap para localização;
- Playwright para testes.

6:00 - 9:00 Demonstração

Sugestão de demo:

1. Login como usuário.
2. Buscar quadra.
3. Abrir detalhe.
4. Reservar horário.
5. Ver código da reserva.
6. Login como proprietário.
7. Ver reserva recebida.
8. Validar código.
9. Abrir tela de BI e mostrar PDF.

10. Login como admin e mostrar validação documental.

9:00 - 11:00 Banco de dados

Explicar entidades:

- usuários;
- proprietários;
- quadras;
- horários;
- reservas;
- documentações;
- notificações;
- administradores;
- logs;
- advertências.

Destacar relacionamentos:

Proprietário -> Quadras -> Horários -> Reservas <- Usuário
Admin -> Documentações / Logs / Controle geral

11:00 - 12:30 Segurança

Explicar:

- JWT;
- bcrypt;
- CPF/CNPJ;
- proteção por perfil;
- uploads limitados;
- logs administrativos;
- bloqueio temporário e banimento.

12:30 - 14:00 Diferenciais

Destacar:

- três perfis completos;
- documentação e aprovação de espaços;
- mapa e geolocalização;
- BI para proprietário;
- notificações e eventos em tempo real;
- advertências para cancelamentos indevidos.

14:00 - 15:00 Conclusão

Fechar com:

- problema resolvido;
- impacto local;
- tecnologias aplicadas;
- evolução futura com pagamento real, app mobile e expansão para outras cidades.

16. Manual de Estudo

16.1 O que preciso decorar

- Nome e objetivo do projeto.
- Três perfis: usuário, proprietário e administrador.
- Tecnologias principais: React, Vite, Node.js, Express, Sequelize, PostgreSQL, JWT, Multer, Playwright.
- Fluxo básico: frontend -> API -> service -> model -> banco.
- Principais entidades: Usuario, Proprietario, Quadra, HorarioDisponivel, Reserva, DocumentacaoLocal, Administrador.
- Regras importantes: documentação aprovada para quadra pública, conflito de reserva, JWT, hash de senha, validação de CPF/CNPJ.

16.2 O que preciso entender

- Por que a regra de negócio fica no backend.
- Como o JWT protege rotas.
- Como o Sequelize representa tabelas e relacionamentos.
- Como a reserva depende de quadra, horário e usuário.
- Como proprietário e administrador têm permissões diferentes.
- Como a validação documental aumenta confiabilidade da plataforma.
- Por que pagamento ainda é uma melhoria futura.

16.3 O que posso consultar

- Nome exato de arquivos.
- Lista completa de rotas.
- Campos específicos de cada tabela.
- Perguntas e respostas da banca.
- Detalhes de bibliotecas secundárias.
- Ordem exata dos imports/componentes.

16.4 Pontos críticos que a banca costuma perguntar

- Segurança de senha.
- Autenticação e autorização.
- Banco de dados e relacionamentos.
- Como evita conflito de reserva.
- Como valida CPF/CNPJ.
- Diferença entre usuário, proprietário e admin.
- O que está implementado e o que é futuro.
- Por que usar determinada tecnologia.
- O que acontece em cancelamentos.
- Como o sistema escalaria para produção.

16.5 Resumo de bolso

PlayArena é uma plataforma web de reservas esportivas.
Frontend: React + Vite.
Backend: Node.js + Express.
Banco: PostgreSQL com Sequelize.
Autenticação: JWT + bcrypt.
Arquivos: Multer.
Localização: ViaCEP + OpenStreetMap/Nominatim.
Tempo real: Server-Sent Events.
Perfis: usuário, proprietário e administrador.
Entidade central: Reserva, ligada a Usuario, Quadra e HorarioDisponivel.
Diferencial: validação documental, gestão administrativa e BI do proprietário.

17. Organização da Apostila e PDF Final

Esta documentação foi organizada como apostila técnica e funcional para uso em:

- estudo individual;
- apresentação de TCC;
- revisão antes da banca;
- base para slides;
- material de apoio durante ensaio de demonstração.

17.1 Como usar este material

1. Leia primeiro a visão geral e arquitetura.
2. Depois estude os fluxos por perfil.
3. Revise banco de dados e APIs.
4. Treine as perguntas da banca.
5. Use o roteiro de 15 minutos para ensaiar a apresentação.

17.2 Sugestão de slides

1. Título e integrantes.
2. Problema.
3. Objetivo.
4. Público-alvo.
5. Arquitetura.
6. Tecnologias.
7. Fluxo do usuário.
8. Fluxo do proprietário.
9. Fluxo do administrador.
10. Banco de dados.
11. Segurança.
12. Demonstração.
13. Funcionalidades implementadas.
14. Melhorias futuras.
15. Conclusão.

17.3 Conclusão técnica

O PlayArena apresenta uma arquitetura coerente para uma plataforma de reservas esportivas. O sistema separa responsabilidades entre frontend, backend e banco de dados, aplica autenticação por perfil, controla regras de reserva, valida documentação de espaços, organiza uploads e oferece painéis específicos para cada tipo de usuário.

Para um TCC, os pontos mais fortes são:

- escopo funcional completo;
- três perfis bem definidos;
- regras reais de negócio;
- uso de banco relacional;
- segurança com JWT e bcrypt;
- integração com APIs externas;
- testes automatizados;
- área de BI para o proprietário.

O principal ponto de evolução é transformar o registro de forma de pagamento em uma integração financeira real, com gateway, confirmação automática, estorno e conciliação.